

LAPORAN PRAKTIKUM
THREADS & CONCURRENCY



Dosen Pengampu : Triawan Adi Cahyanto, M.K

Di susun Oleh:

Muhammad maulana wiramatajaya 2410651006

PROGRAM STUDY TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2024

1. KALKULATOR PARALEL

A. Buat file baru dengan cara ;

```
nano kalkulator.c
```

B. Masukkan kodingan bawah kedalam file `kalkulator.c`

```
GNU nano 4.8                                kalkulator.c                                Modified
#include <stdio.h>
#include <pthread.h>

double a,b;
double h_tmbh,h_krng,h_kali,h_bagi;

//tambah
void* tmbh(void* arg){
    h_tmbh = a+b;
    pthread_exit(NULL);}

//kurang
void* krng(void* arg){
    h_krng = a-b;
    pthread_exit(NULL);}

//kali
void* kali(void* arg){
    h_bagi = a*b;
    pthread_exit(NULL);}

//bagi
void* bagi(void* arg){
    h_bagi = a/b;
    pthread_exit(NULL);}

}

int main(){

pthread_t t1,t2,t3,t4;

//mengambil input user
printf("masukan angka 2 : ");
scanf("%lf %lf",&a,&b);

//membuat tread
pthread_create(&t1,NULL,tmbh,NULL);
pthread_create(&t2,NULL,krng,NULL);
pthread_create(&t3,NULL,kali,NULL);
pthread_create(&t4,NULL,bagi,NULL);

//menunggu thread bekerja

^G Get Help      ^O Write Out    ^W Where Is     ^K Cut Text     ^J Justify
^X Exit          ^R Read File    ^_ Replace      ^U Paste Text   ^T To Spell
```

```
//menunggu thread bekerja
pthread_join(t1, NULL);
pthread_join(t2, NULL);
pthread_join(t3, NULL);
pthread_join(t4, NULL);

printf("\n == HASIL == \n");
printf("tambah : %.2lf\n", h_tmbh);
printf("kurang : %.2lf\n", h_krng);
printf("kali : %.2lf\n", h_kali);
printf("bagi : %.2lf\n", h_bagi);
return 0;

}

^G Get Help      ^O Write Out     ^W Where Is      ^K Cut Text      ^J Justify
^X Exit          ^R Read File     ^\ Replace       ^U Paste Text    ^T To Spell
```

Untuk menyimpan kodingan tekan **ctrl + o** lalu **Enter** Lalu tekan **ctrl + x** **ctrl** Untuk keluar kodingan.

C. Lalu Compile dengan cara memasukan kode di bawah kedalam terminal :

```
gcc -o kalkulator kalkulator.c
```

D. Lalu Running dengan cara memasukan kode di bawah kedalam terminal :

```
./kalkulator
```

E. Masukan 2 angka lalu enter maka hasilnya akan seperti di bawah ini :

```
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_3/tugas$ ./kalkulator
masukan angka 2 : 10 5

== HASIL ==
tambah : 15.00
kurang : 5.00
kali : 0.00
bagi : 2.00
wiramatajaya@LAPTOP-7B38G7J9:~/pertemuan_3/tugas$
```

2. FILE PROCESSING

A. Buat file baru dengan cara ;

```
nano file_processing.c
```

B. Masukkan kodingan bawah kedalam file **kalkulator.c**

```
GNU nano 4.8                                file_processing.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <time.h>

#define MAX_SIZE 1000000 // batas ukuran file teks

// Struktur data untuk thread
typedef struct {
    char *text;
    int mulai;
    int selesai;
    int htng;
} ThreadData;

// menghitung jumlah kata di sebagian teks
void* htng_words(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    int htng = 0;
    int in_word = 0;

    for (int i = data->mulai; i < data->selesai; i++) {
        char c = data->text[i];
        if (c == ' ' || c == '\n' || c == '\t' || c == '\r')
            in_word = 0;
        else if (!in_word) {
            in_word = 1;
            htng++;
        }
    }

    data->htng = htng;
    pthread_exit(NULL);
}

// menghitung kata single thread
int htng_words_single(char *text, int length) {
    int htng = 0, in_word = 0;
    for (int i = 0; i < length; i++) {
        char c = text[i];
        if (c == ' ' || c == '\n' || c == '\t' || c == '\r')
            in_word = 0;
        else if (!in_word) {
            in_word = 1;
            htng++;
        }
    }
    return htng;
}
```

```

// MULTI THREAD
pthread_t t1, t2;
ThreadData d1 = {text, 0, (int)(len / 2), 0};
ThreadData d2 = {text, (int)(len / 2), (int)len, 0};

clock_t mulai_mt = clock();
pthread_create(&t1, NULL, htng_words, &d1);
pthread_create(&t2, NULL, htng_words, &d2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
clock_t selesai_mt = clock();

int total_mt = d1.htng + d2.htng;
double time_mt = (double)(selesai_mt - mulai_mt) / CLOCKS_PER_SEC;

//SINGLE THREAD
clock_t mulai_st = clock();
int total_st = htng_words_single(text, len);
clock_t selesai_st = clock();
double time_st = (double)(selesai_st - mulai_st) / CLOCKS_PER_SEC;

// hasil
printf("\n=== HASIL PERHITUNGAN ===\n");
printf("Jumlah kata (multi-thread): %d\n", total_mt);
printf("Jumlah kata (single-thread): %d\n", total_st);
printf("\nWaktu eksekusi multi-thread: %.6f detik\n", time_mt);
printf("Waktu eksekusi single-thread: %.6f detik\n", time_st);

free(text);
return 0;
}

```

[^]G Get Help [^]O Write Out [^]W Where Is [^]K Cut Text [^]J Justify [^]C Cur Pos
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste Text [^]T To Spell [^]_ Go To Line

Untuk menyimpan kodingan tekan **ctrl + o** lalu **Enter** Lalu tekan **ctrl + x** **ctrl** Untuk keluar kodingan.

C. Lalu Compile dengan cara memasukan kode di bawah kedalam terminal :

```
gcc file_processing.c -o file_processing -lpthread
```

D. Lalu Running dengan cara memasukan kode di bawah kedalam terminal :

```
./file_processing
```

E. Masukan 2 angka lalu enter maka hasilnya akan seperti di bawah ini :

```

Panjang file: 28 karakter

=== HASIL PERHITUNGAN ===
Jumlah kata (multi-thread): 4
Jumlah kata (single-thread): 3

Waktu eksekusi multi-thread: 0.000509 detik
Waktu eksekusi single-thread: 0.000000 detik

```

3. PRODUCER CONSUMER

A. Buat file baru dengan cara ;

```
nano producer_consumer.c
```

B. Masukan kodingan bawah kedalam file **producer_consumer.c**

```
GNU nano 4.8 producer_consumer.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int count = 0; // jumlah item dalam buffer

pthread_mutex_t mutex;
pthread_cond_t full, empty;

void* producer(void* arg) {
    while (1) {
        int item = rand() % 100;
        pthread_mutex_lock(&mutex);

        while (count == BUFFER_SIZE) {
            printf("Buffer penuh! Producer menunggu...\n");
            pthread_cond_wait(&empty, &mutex);
        }

        buffer[count++] = item;
        printf("Producer menghasilkan: %d (isi buffer: %d)\n", item, count);

        pthread_cond_signal(&full);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
}

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);

        while (count == 0) {
            printf("Buffer kosong! Consumer menunggu...\n");
            pthread_cond_wait(&full, &mutex);
        }

        int item = buffer[--count];
        printf("Consumer memproses: %d (isi buffer: %d)\n", item, count);

        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&mutex);
        sleep(2);
    }
}

int main() {
    pthread_t prod, cons;

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&full, NULL);
    pthread_cond_init(&empty, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&full);
    pthread_cond_destroy(&empty);

    return 0;
}
```

Untuk menyimpan kodingan tekan **ctrl + o** lalu **Enter** Lalu tekan **ctrl + x ctrl** Untuk keluar kodingan.

C. Lalu Compile dengan cara memasukan kode di bawah kedalam terminal :

```
gcc producer_consumer.c -o producer_consumer -lpthread
```

D. Lalu Running dengan cara memasukan kode di bawah kedalam terminal :

```
./producer_consumer
```

E. Masukan 2 angka lalu enter maka hasilnya akan seperti di bawah ini :

```
Buffer kosong! Consumer menunggu...
Producer menghasilkan: 83 (isi buffer: 1)
Consumer memproses: 83 (isi buffer: 0)
Producer menghasilkan: 86 (isi buffer: 1)
Consumer memproses: 86 (isi buffer: 0)
Producer menghasilkan: 77 (isi buffer: 1)
Producer menghasilkan: 15 (isi buffer: 2)
Consumer memproses: 15 (isi buffer: 1)
Producer menghasilkan: 93 (isi buffer: 2)
Producer menghasilkan: 35 (isi buffer: 3)
Producer menghasilkan: 86 (isi buffer: 4)
Consumer memproses: 86 (isi buffer: 3)
Producer menghasilkan: 92 (isi buffer: 4)
Consumer memproses: 92 (isi buffer: 3)
Producer menghasilkan: 49 (isi buffer: 4)
Producer menghasilkan: 21 (isi buffer: 5)
Consumer memproses: 21 (isi buffer: 4)
```

```
nano file_processing.c
```